Data Pull Collector Communication Parallelization

Design Document

Contents

References 5
Introduction 6
Background 6
Design Goals 6
Project Scope 6
Audience 7
Requirements 8
Requirement Discussion 8
Derived Requirements 8
Prerequisites 8
Architecture 9
Introduction 9
Storage Objects 9
Communication (Pulling) 9
Storage 9
Frequency Levels 10
Scalability 10
Pulling 10
Storing 10
High Level Design 12
Storage Objects 12
Introduction 12
Serialization 12
Store Routine 13
Storage Concurrency 13
Retries and Idempotence 13
Collector Communication 14
Introduction 14
Operations 14
Checkpointing 14
Collector Database 15
Introduction 15
Storage Tables 15

Checkpointing 15
Storage Data Retention 16
Low Level Design 17
Introduction 17
Twisted 17
StorageTaskManager 19
Overview 19
Storage Tasks Buffer 19
Task Distribution 20
Storage Task Results 21
CollectorManager 22
Overview 22
Collector State Machine 22
Collector Refresh 23
Pull Requests 24
Checkpoints 24
CollectorProtocol 26
Overview 26
Operations 26
Error Handling 27
ProcessPool 28
Overview 28
StorageProcessProxyProtocol 29
Overview 29
Spawning 29
Respawning 29
Communication Protocol 29
StorageTaskBlock 31
Storage Pool Processes 32
Overview 32
StorageProcessProtocol 32
Configurables 33
Testing 35
Unit Testing 35

QA Testing 35
Functional Testing Strategy 35
Performance and Scale Testing Strategy 35
Instrumentation 37
Introduction 37
Optimization for Data Pull Project 37
Polling for Metrics 37
Basic Usage 37
Data Pull Metrics Groups 38
Example Usage 38
Schema 38
Troubleshooting 42
Miscellaneous 43
Open Issues 43
Glossary 44

References

Project Page on Confluence:

<Link>

Requirements document linked on Confluence Page:

<Link>

Link to this document on Confluence

<Link>

- http://twistedmatrix.com/documents/current/core/howto/testing.html

Introduction

This design document describes the implementation of the requirements described in Data Pull.

Background

The present implementation of data pull handles collector communication in a highly sequential manner. Only a single pull from a collector can happen at any time and those pulls are only triggered after the storage tasks have all been distributed to the storage pool. In addition to this, the main process/thread context is responsible for more than connection management and pulling from collectors – it also handles task distribution and process pool management.

This presents an obstacle for scaling distributed deployments horizontally with more collectors. With the advent of SSD storage solutions for our customers, the biggest bottleneck is now producing work for starved storage processes. Data Pull's overall performance is also too sensitive to isolated chronic communication deficiencies that can arise due to network or collector performance issues.

Design Goals

There are two primary goals in this project:

- Increase EM7's horizontal scalability by making data pull capable of accommodating more collectors in the system
- Limit the overall performance impacts due to communications problems with a subset of collectors

Other goals:

- Do not starve the storage process pool when there is work to be done
- Write testable code – modularize the code well enough to be conducive for coverage tests
- With the process pool, replace our use of fork/no-exec by way of the multiprocessing module with spawned processes (fork/exec)
- Augment instrumentation
- Add more configurable settings where needed to enable field changes for varying performance concerns
- Develop expertise in using the Twisted framework

Project Scope

The scope of this project has evolved into a complete re-design and rewrite for all three major components of Data Pull: puller, storage pool, task broker.

The architecture will remain untouched.

Audience

TODO

Requirements

Requirement Discussion

There is a single requirement: allow concurrent collector communication in Data Pull.

Derived Requirements

- Must limit resource usage
- Instrumented
- Pulling should be "fair"
- Storage Object Retries on timeouts, and DB contention errors
- Greedily use resources allocated
- Configurables
- Process reqs
    - Killable
    - Timeouts

Prerequisites

TID 15550 – silo_metrics optimization
TID 16156 – Python unit testing

Architecture

Introduction

Data Pull is a critical process that runs on the central database appliance for distributed EM7 systems. This process is responsible for polling collector units for *storage objects* (messages) that are to be communicated back to the central database. In this regard, Data Pull has an opposite and complementary function to the Config Push process.

These storage objects may contain data such as alerts, collected data, and system log messages.

Storage Objects

Collectors share discreet messages via *storage objects*. Storage objects are queued in a table local to the collector. Data Pull pulls contiguous blocks of storage objects at a time. It tracks progress by persistently setting a checkpoint in a table in the central database.

All Storage Objects have a corresponding *store routine*. The store routine defines how to act on the data contained in the message. This most often entails inserting or modifying data on the central database.

Communication (Pulling)

Our system architecture only allows for central database appliances to connect to collector appliances via a single port. In addition to using MySQL database servers for data persistence, we rely heavily on them as middleware to facilitate inter-process communication, synchronization, and configuration. This reliance extends to appliance-to-appliance communication. As such, Data Pull polls collectors for new information (collected data, events, etc.) by connecting to and querying against a collector's MySQL database server instance.

Collectors are not provided a means of signaling back to central database appliances other than by defined storage objects.

Storage

Once Data Pull has pulled storage objects from a collector, it must then act upon them. This almost always results in changes in the central database. This general case is the reason why this step is considered "storage" and lends the name "storage objects" to the messages.

Frequency Levels

The frequency level determines the latency in communicating a storage object back to the central database. Higher frequencies result in higher effective priorities for storage objects.

Data Pull has three frequency levels that define a minimum polling frequency: High Frequency (15 seconds), Medium Frequency (60 seconds), and Low Frequency (300 seconds). This is currently supported by having 3 separate instances of Data Pull; one for each priority. These instances are often referred to with their abbreviated names: "HF Pull", "MF Pull", "LF Pull".

Alerts fall under the high frequency category. Collected data (performance, configuration, journals, etc.) fall under the medium frequency.

Since collected data represents the bulk of the load generated by collectors, performance issues are most evident in medium frequency data pull.

Scalability

There are two scalability concerns: pulling and storing.

Pulling

There are three problems to address with pulling:

- Horizontal scaling – managing connections to multiple collectors
- "Slow collectors" – performance varies for many reasons including network performance and collector database performance (due to provisioning and/or collection load)
- Disruption of network communication

Storing

Storage entails executing arbitrary storage object store routines. It is possible for storage objects to have a CPU-intensive workloads. In practice however, most work is offloaded to the central MySQL database. Since the bulk of storage work is handled in the database, MySQL performance is the greatest potential bottleneck.

Potential database bottlenecks:

- Poorly crafted table indexes or queries
- Storage I/O
- System resource contention
- DB-level contention

High Level Design

Storage Objects

Introduction

Each discreet message of communication is represented in backend Python code as a storage object.

All storage object classes are derived from `storage/storage_objects.py`'s `base_storage` class. This class provides two context-specific interfaces: the Collector Store Routine (`collector_store()`), and the central database's Store Routine (`store()`). The former is only ever called on collectors while the latter is only ever called in a central database appliance.

The exemption is on all-in-one appliance where the store routine is invoked from `collector_store()` as there is only a single appliance.

The Collector Store Routine serializes the storage object and queues it in the appropriate in_storage table.

Serialization

The base class implements the Collector Store Routine. On distributed systems it stores a serialized version of the storage object in the collector's database. The derived class must specify the priority level of the storage object (HF, MF, LF) as this determines which queue the storage object will be appended to.

Storage objects are serialized as Python pickles before being inserted into the database.

Pickling only serializes data, not executable code. When a pickle is de-serialized, a new instance of the object is created that is initialized with the same state as the original storage object. To do so, pickle relies on preserved path information to resolve the class definition in the unpickling environment.

This requires that identical class definitions for storage objects (and by extension, paths to storage object modules) must be available both on the central database appliance and on all collector appliances.

Implementors should also be wary of pickling objects that contain resource handles. The pickle protocol allows an object's class to define what exactly is serialized when

the object is pickled. See http://docs.python.org/2/library/pickle.html#the-pickle-protocol.

Pickles are sensitive to versioning issues – these come to light during patch timeframes when build versions can be out of sync among appliances or when pre-patch versions of storage objects are queued before data pull is restarted using the new definitions. Developers should try their best to avoid handle this condition.

Store Routine

The store routine is implemented by derived classes of base_storage. The implementer of the storage object must decide how the central database will act on the data communicated from the collector. This store routine is invoked by Data Pull after pulling and de-serializing the storage object.

Storage Concurrency

Excecution of storage object store routines may be parallelized. To reduce contention in database operations, storage objects may have a synchronization/serialization key defined. This allows Data Pull to work on multiple unrelated storage tasks in parallel while preventing parallelization of certain groups of tasks.

Retries and Idempotence

Data Pull may retry storing a storage object if storage fails due to a MySQL Lock Wait Timeout or Deadlock, or if it is due to a Data Pull-level timeout. Since transactions are rarely used in the backend, there is a chance that failed storage object may have made changes to the database. Also, retries can potentially result in out-of-sequence storage. It is important that storage objects should aim to be idempotent – subsequent executions should always provide the same result. This isn't always feasible for cases where counters must be incremented.

Collector Communication

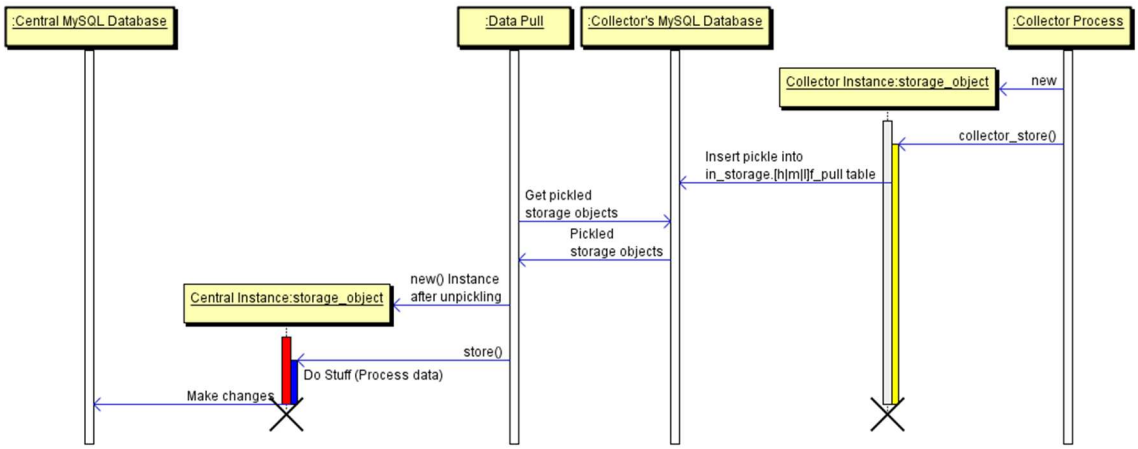
Introduction

Backend processes on collectors instantiate and initialize storage objects with data that it wishes to communicate back to the central database.

Once the storage object is created, it is then serialized as a Python pickle and queued up in the collector's database in the designated storage table.

Data Pull then pulls the data, de-serializes it, and then executes the store routine.

The following sequence diagram illustrates the life of a storage object.



Operations

Synchronization: Polling the collector for the maximum and minimum sequence IDs in the storage table along with the last remote checkpoint. This information is used for ensuring checkpoint consistency and for calculating the queue size.

Pulling: Query for a block of storage objects from the collector's storage object queue.

Remote Checkpointing: Update the checkpoint for progress.

Checkpointing

Data Pull maintains a central table that tracks pull progress for each collector for each pull frequency.

master.datapull_checkpoints

Field	Type	Collation	Attributes	Null	Default	Extra
collector_id	int(10)		UNSIGNED	No		
frequency	enum('high_freq', 'med_freq', 'low_freq')	latin1_swedish_ci		No		
seq	int(11)		UNSIGNED	No		

Data Pull will always favor entries in this table over the collector's checkpoint table. These values should not be manually edited while Data Pull runs.

Collector Database

Introduction

Collectors queue pickled storage objects in a table in its MySQL database.

The collector also hosts a table used for checkpointing pulling progress. It is Data Pull's responsibility to maintain the state of that table.

Storage Tables

There is a storage table for each pull priority in the in_storage database. The tables are: hf_storage, mf_storage, and lf_storage. All tables share the same schema definition.

in_storage.[h|m|l]f_pull

Field	Type	Collation	Attributes	Null	Default	Extra
seq	int(11)		UNSIGNED	No		auto_increment
did	int(11)		UNSIGNED	No		
collection_time	datetime			No		
data	mediumblob		BINARY	No		

For data pruning purposes, this table is partitioned by collection_time.

Checkpointing

A single table on each collector is used for tracking Data Pull's progress for all three priority levels.

in_storage.data_storage_last_collect

Field	Type	Collation	Attributes	Null	Default	Extra
seq	int(11)		UNSIGNED	No		
frequency	enum('high_freq', 'med_freq', 'low_freq')	latin1_swedish_ci		No		

The frequency column serves as the key for each priority level. There is a single row per key.

Previously, Data Pull used these tables to track its progress through the in_storage queues. Now this table is limited to two purposes:

- Provides the last checkpoint value to data pull only when no central checkpoint is available (this should only happen as a migration step when moving to the updated version of data pull)
- Marks all storage objects older than this checkpoint is eligible for pruning by daily maintenance if they are also older than retention setting.

Storage Data Retention

The storage table data is pruned by a local instance of the Daily Maintenance process in the "collector_dbcleanup" maintenance task.

The retention setting in days is maintained in the master.system_settings_core table in the records_cu_after_pull field.

The pruning task is careful to not prune any rows that exist after the last checkpoint.

Low Level Design

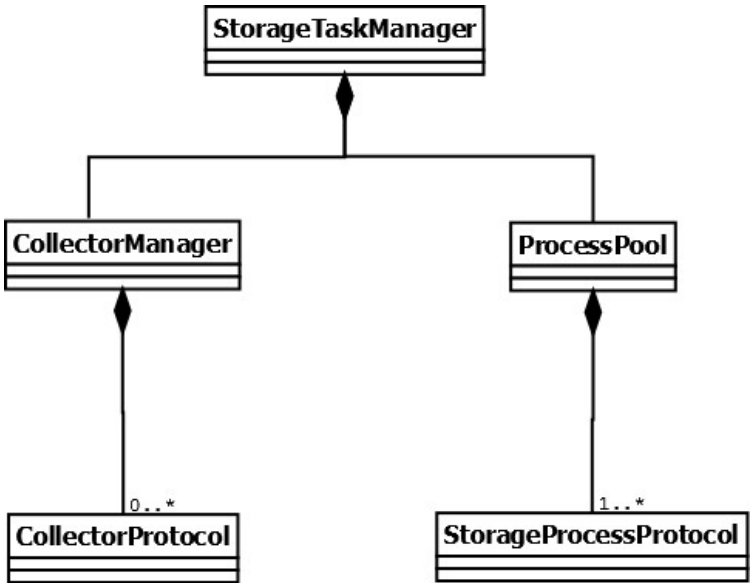
Introduction

We will use the Twisted framework to facilitate an event-driven approach to handling concurrency.

Data pull's object model is broken into three major top-level components to correspond to the three major responsibilities:

- CollectorManager: Pulling (producer)
- ProcessPool: Storing (consumer)
- StorageTaskManager: Task distribution & enforcing resource limits (pausing the producer)

The following illustrates the hierarchical relationships:



Twisted

We use Twisted to drive an asynchronous event loop.

There are two limited resource pools for deferring work – the thread pool and the storage process pool. We do not have a naïve producer-consumer model however as some coordination is required for the task distribution and there is some bidirectional communication between both ends.

Twisted's mature and robust framework is attractive for handling the multiple types of events that would arise in data pull.

Any potentially blocking operation is handled asynchronously. In this model, the client of an interface will receive a Twisted Deferred object back from a request. The client can then attach callback and errbacks (callbacks for error handling in Twisted parlance) to the Deferred that are triggered when the requested operation completes.

Those long running operations can be deferred to a thread, a subprocess, or some non-blocking library or system call. This allows the event loop to continue operating on the next callback in its queue in a responsive manner.

In Twisted, the Reactor is responsible for driving the event loop and triggering the initial callbacks in the callback chains. For our project, we use the epoll implementation of the Reactor.

StorageTaskManager

Overview

The StorageTaskManager can be seen as the central task broker. It is responsible for directing the CollectorManager to pull blocks of storage tasks and for distributing those tasks to the ProcessPool for storage. StorageTaskManager initiates all interaction with the other two components by issuing asynchronous requests to which it attaches callbacks for responses.

As blocks of storage tasks become available from the CollectorManager, StorageTaskManager will buffer what it cannot immediately distribute to the ProcessPool.

StorageTaskManager is mindful of serialization keys defined for storage tasks. To prevent parallelization of certain groups of tasks, it will push tasks to a temporary deferred queue if its serialization key is currently active.

Once tasks complete, StorageTaskManager must handle task results. Instrumented counters and system log messages are used to track failure conditions. It also allows for retries of storage objects that fail due to subset of conditions.

Storage Tasks Buffer

The storage tasks buffer is composed of:

- All storage tasks currently active (actively being worked on by a pool process)
- All storage tasks queued in the main queue
- All storage tasks queued in a deferred queue

The size of this buffer is limited by the `outstanding_storage_tasks_limit` configuration option. This limit is defined in terms of storage objects.

To prevent overruns of the buffer size, StorageTaskManager will reserve `pull_block_size` number of storage objects for each outstanding pull request.

Active Tasks

Active tasks are all tasks currently distributed to the ProcessPool. These are still alive in the buffer since tasks may need to be retried. There are up to `num_storage_procs` active tasks at a time.

Main Queue

The main queue is implemented as a sort-stable priority queue derived from the PriorityQueue implementation in the Queue standard library module.

Sort-stable means that for items that are in the same priority level, the ordering is based on the time of push.

There are only 2 priority levels. The highest is for first attempts. The lowest is for retries.

(TODO) Considerations for future design changes:

- The current design does not handle aging of lower priority tasks to prevent starvation
- Also, make own implementation using the heap_q module directly since synchronization (from Queue.queue) is not required
- Using the priority queue could allow handling all three pull frequencies in a single data pull instance

Deferred Queues

As tasks are distributed to the ProcessPool, their serialization keys are registered. Storage tasks are pushed to deferred queues when the StorageTaskManager sees that the task's serialization key is currently registered. This allows data pull to serialize tasks that are order-sensitive or are likely to cause contention in the MySQL database.

There are up to `num_storage_procs` deferred queues at any given time – 1 for each registered serialization key.

Task Distribution

When a block of storage tasks becomes available, for each storage task, Storage Task Manager will:

1. See if there are any free pool processes (based on number of active storage tasks). If not, then push the task to the main queue, else…
2. See if the task's serialization key is currently registered. If so, then push to the associated deferred queue, else…
3. Register the serialization key and submit the task directly to the ProcessPool

When a task completes and thus a pool process is freed, StorageTaskManager will:

1. Use the serialization key of the completed task to check if there are any other tasks queued in the associated deferred queue. If so, pop a task and submit it to the pool, else de-register the serialization key and…
2. Check the main queue to see if a task is queued. If not, then do nothing, else…
3. If that popped task's serialization key is currently registered, push it to the associated deferred queue and repeat step 2, else…
4. Register the serialization key and submit the task directly to the ProcessPool

While distributing newly pulled tasks, StorageTaskManager registers a callback with the StorageTask's deferred to increment the StorageBlock's "tasks done" counter so that the CollectorManager can track checkpoints.

Storage Task Results

As storage tasks complete, StorageTaskManager handles the results. The callback chain is continued in case if there are any callbacks attached to the task's deferred (used by the CollectorManager to track completion of storage blocks).

When all attempts have been exhausted, the storage object is dropped out of the buffer to potentially allow another pull request.

Some failures are retryable – limited by the `num_storage_retries` configuration option. When a task is retried, it is pushed onto the main queue at a lower priority level if there is no free pool process. When all retries have been exhausted and the final failure condition is due to a retryable error, a NOTICE-level message will be logged to the system logs.

Non-retryable failures will be logged as MAJOR-level messages in the system logs

Results as seen in System Messages:

- OK – successful storage
- Retryable Failures (logged as INFO-level messages only if all retries exhausted)
    - DEADLOCK – MySQL Deadlock encountered
    - LOCK WAIT TIMEOUT – MySQL Lock Wait Timeout encountered
    - TIMEMOUT – Storage object did not complete within `store_timeout` seconds
    - MYSQL CONNECTION LOST – Pool process lost connection to the local MySQL database during storage
    - UNEXPECTED EXIT – Pool process exited for an unanticipated reason (user SIGTERMed?)
    - UNEXPLAINED FATAL ERROR – An unanticipated condition was encountered, so we should retry the storage object to be safe
- Non-Retryable Failures (logged as ERROR-level messages)
    - ERROR – exception raised during store routine
    - UNPICKLE ERROR – The storage object was corrupted or truncated when stored on the collector database
    - FATAL ERROR – Pool process exited due to a fatal signal (SIGSEGV, SIGABRT, SIGFPE, SIGBUS, SIGILL)
    - COMMUNICATION ERROR – internal communication error between the main data pull process and the pool process

CollectorManager

Overview

The CollectorManager drives collector interactions in support of fulfilling pull requests from the StorageTaskManager.

At regular intervals corresponding to the pull frequency (e.g. every seconds for MF Pull), CollectorManager performs a "Collector Refresh". It will poll the central database for a list of available collectors (and message collectors). Based on that list, it will then instantiate CollectorProtocol instances for each collector seen for the first time. CollectorProtocol instances represent the valid operations that can be performed on a collector. CollectorProtocol instances that are currently idle ("caught up") are then eligible for a Sync operation which may lead to it being eligible for a Pull operation.

CollectorManager will then drive the state machine for collector interaction. The operations are: Connect, Sync, Pull, Checkpoint.

CollectorManager parallelizes interaction with CollectorProtocol instances by using deferring requests to the Twisted reactor's central thread pool. Only a single request may be issued at a time for each CollectorProtocol instance.

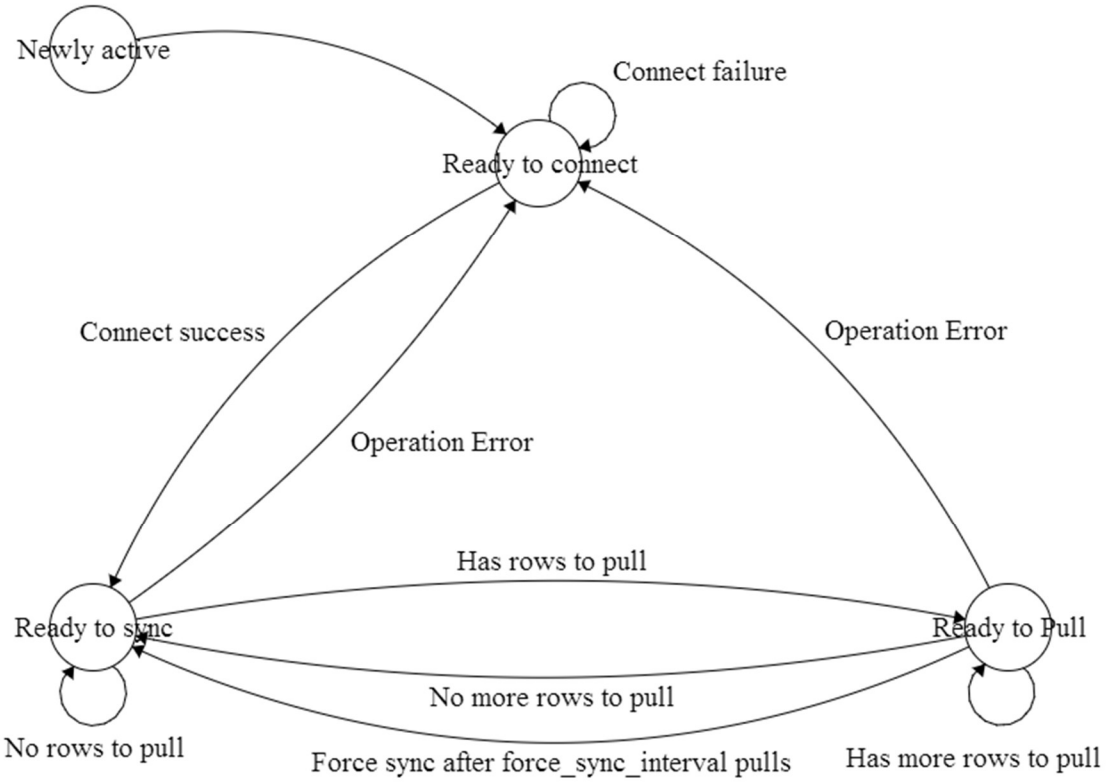
Collector State Machine

CollectorProtocol instances can only perform a single operation at a time. To accommodate this, CollectorManager manages a state machine for each collector. It maintains the following queues to manage state transitions:

- Collectors-to-connect
- Collectors -to-sync
- Collectors -to-pull
- Inactive Collectors

All newly active CollectorProtocol instances are pushed to the collectors-to-connect queue.

This abridged finite state machine diagram mostly describes the state and transitions for a CollectorProtocol instance:



The diagram leaves out remote checkpoint operations for simplicity. Remote checkpoint requests are asynchronous (done every `remote_checkpoint_interval` minutes). CollectorProtocol has built-in serialization capabilities to ensure that a remote checkpoint operation will not run in parallel with any other operation. However, a failure will result in the collector being placed in the Ready To Connect state.

If a collector is noted to be inactive (unavailable) during a Collector Refresh, then the collector is removed from the state queues and held in an inactive set. The CollectorProtocol instance is maintained since it tracks pull progress. Once the collector is available again, it will be moved to the Ready To Connect queue.

Collector Refresh

Collector Refreshes are responsible for maintaining the list of active collectors and for triggering syncs (and thus pulls).

Collector Refreshes are regularly scheduled according to the frequency of the data pull instance.

This operation first polls for a list of available collectors on the system. A separate backend process (Collector Task Manager) is responsible for managing the availability state of collectors.

It then performs set operations to determine:

- Set of newly active collectors
- Set of newly inactive collectors

Newly active collectors are placed in the Collectors-to-connect queue. If there is no corresponding CollectorProtocol instance, one is created.

Newly inactive collectors are popped from all state queues and placed in the Inactive list.

Connection and sync requests are then deferred (queued) to the thread pool. If a sync operation fails, a single reconnect attempt is allowed immediately for this sync interval. If that also fails, then the collector will be pushed on the Collectors-to-connect queue and will have to wait until the next Collector Refresh for another connection attempt.

Collectors that are in the Collectors-to-pull queue will effectively remain in the same state – they will not be subject to an unnecessary sync operation.

Pull Requests

The StorageTaskManager may queue up requests by making several pull requests before pull results are provided.

These pull requests are absent of context. They serve as directives to CollectorManager. CollectorManager can service those requests with pull results from any collector.

At any given time, there are only up to as many CollectorProtocol pull requests as queued pull requests from the StorageTaskManager. CollectorManager selects the next collector to pull from by popping from the Collectors-to-pull list in FIFO order.

Pulls are done in blocks of `pull_block_size`. StorageTaskManager must expect that it is possible for a block of storage tasks in a pull result consists of less than a full block size.

Checkpoints

CollectorManager manages the checkpoints for individual collectors.

When a CollectorProtocol object is instantiated, CollectorManager provides the last known central checkpoint for that collector.

As blocks of storage tasks are pulled, it maintains a list of outstanding blocks (storage not complete) for each collector. When a block is complete, it then checks if there are any blocks preceding it that are not yet complete. If that block and any preceding ones are complete, then the central checkpoint is set immediately to the last ID in the last complete block.

Remote checkpoints are done less frequently to lessen the impact on pull performance since these checkpoints are only used for data pruning. Remote checkpoints are scheduled every `remote_checkpoint_interval` minutes.

CollectorProtocol

Overview

CollectorProtocol instances:

- Abstract away the data access portion of collector operations
- Maintain the position of the next pull for this collector

The CollectorManager tracks the state of these instances and drives state transitions.

Operations

Requests to run operations are deferred to the central thread pool owned by the Twisted Reactor.

CollectorProtocol natively serializes requests to prevent concurrent use of the MySQL database connection. This was taken advantage of to allow checkpoint requests to be made at any time regardless of what state the CollectorProtocol instance is in.

Connect

Connect requests must be performed before any other operation is possible. Open connections are held for as long as possible.

Connect operations are subject to the `collector_connect_timeout` timeout setting.

Sync

Sync operations are used to:

- Poll for size of storage object backlog
- Update collector statistics
- Monitor for discrepancies in sequence IDs reported in the storage object queue compared to where the CollectorProtocol instance believes the next pull should start

The callback from the sync request returns the number of rows to pull.

Subject to the `collector_read_timeout` timeout setting.

Pull

Pull operations try to pull up to `pull_block_size` rows from the storage object queue.

The callback from the pull request returns the list of pulled rows and a Boolean indicating whether or not it believes there are more rows to pull.

Subject to the `collector_read_timeout` timeout setting.

Checkpoint

Checkpoint operations set the remote checkpoint on the collector. This checkpoint is used by the collector to determine what data is safe to prune from the storage object queue.

Subject to the `collector_read_timeout` timeout setting.

Error Handling

There are four classes of errors that can arise from collector operations:

- Connection Lost
- Connection Error
- Operational Error
- Unhandled Exception

The first two are non-exceptional and considered to be expected as possible due to network events. These are logged as debug-level messages.

The second two are truly exceptional and should be brought to the user's attention. These are logged as MAJOR-level messages in the system logs.

Operational Errors should only occur due to unusual circumstances arising from the collector's MySQL database (e.g. table corruption, unexpected schema change, etc). Any such cause should be investigated ASAP.

Unhandled Exceptions are due to a coding error in the underlying collector operations data access layer. These should be non-existent.

ProcessPool

Overview

The ProcessPool maintains a list of StorageProcessProxyProtocol instances which serve as proxies for pool processes.

Up to `num_storage_procs` pool processes can be spawned at any given time.

The ProcessPool allows the StorageTaskManager to asynchronously submit tasks to be distributed to the pool processes as they become ready or complete their last submitted tasks.

Driver code is self-contained in the StorageProcessProxyProtocol class, so the responsibilities of ProcessPool are fairly limited.

StorageProcessProxyProtocol

Overview

StorageProcessProxyProtocol instances handle communication with a single pool process (storage process) at a time. Communication is done via pipes to the pool process's stdin and stdout pipes. Diagnostic error information is collected via the stderr pipe.

StorageProcessProxyProtocol can be considered as a pool process "slot" in that they live for the entire life of data pull but their associations with actual subprocesses will change with time. Because of this, this class blends both the Protocol and Factory concepts in Twisted into a single class since the proxy is long-lived.

Spawning

When the pool process dies or exits, StorageProcessProxyProtocol will automatically spawn a new `data_pull_pool_proc.py` process.

Respawning

As a safeguard against memory leaks, the storage process is killed and restarted after `storage_process_refresh_interval` storage tasks are complete.

Communication Protocol

Communication is done via the pool process's stdin and stdout pipes. StorageProcessProxyProtocol uses 32 bit netstrings to send storage tasks via the pool process's stdin pipe.

There are two sets of requests/responses used for communication:

- Setup – subject to a hard coded initialization timeout
  - Request – send configuration parameters to the pool process
  - Response – MySQL connection ID for connection used in the pool process
- Storage Task – subject to a timeout of `store_timeout` seconds
  - Request – send the pickled storage object
  - Response – StorageTaskResult instance

Diagnostic Information

The pool process will favor embedding diagnostic information in StorageTaskResult object communicated back to data pull. However there are some cases where it is not possible (such as when a segfault is encountered). For these cases, the pool process will communicate as much diagnostic information as possible via stderr.

StorageTaskBlock

Container object for a block of storage objects.

Responsibilities:

- Maintain a task completion counter
- Callback on the block complete deferred to notify when at least 1 attempt has been made to store each storage object

Interface:

- task_done() – increments completion counter and may trigger a callback on the block complete deferred

Storage Pool Processes

Overview

Storage Pool Processes are subprocesses spawned by StorageProcessProxyProtocol instances. These accept pickled storage objects, unpickles them, and then executes the store routine.

Instances are spawned via the `/usr/local/silo/proc/data_pull_pool_proc.py` launch script and exist as children of the data pull instance.

Twisted is used here as well to facilitate an event loop to handle communication with the main data pull process.

StorageProcessProtocol

Handles connections over the stdin/stdout/stderr pipes. Unpickles communicated storage objects and executes their store routines.

Configurables

The following are optional configuration directives for the [**DATAPULL]** section of **/etc/silo.conf**. Previously, the configuration options were in the [LOCAL] section.

Configuration Option	Default	Unit
collector_connect_timeout	5	seconds
collector_read_timeout	10	seconds
collector_thread_pool_size	8	threads
force_sync_interval	5	pulls
instrumentation_relay	twisted	twisted | synchronous | off
log_storage	0	0 | 1
memory_limit_mb	1024	MB
num_storage_procs	number of cpus to a max of 16	processes
num_storage_retries	2	storage object retries
outstanding_storage_tasks_limit	100000	storage objects
pull_block_size	5000	storage objects
remote_checkpoint_interval	30	minutes
storage_process_refresh_interval	1000000	storage objects
store_timeout	60	seconds

collector_connect_timeout – (DEFAULT: 5) Timeout in seconds for an attempted connection to a collector's MySQL database

collector_read_timeout - (DEFAULT: 10) Timeout in seconds for a read from a collector's MySQL database in response to a query. The MySQL client library hardcodes 3 retries – so the effective read timeout is actually 3 times this setting.

collector_thread_pool_size - (DEFAULT: 8) Maximum number of threads that will get created for collector communication

force_sync_interval – (DEFAULT: 5) Number of pulls before a sync operation is forced to keep the estimated rows behind metric up to date. This most commonly applies to "catch up" scenarios where a collector a sync (due to a refresh) reveals that there are multiple blocks to pull

instrumentation_relay – (DEFAULT: twisted) Relay implementation used for reporting instrumentation data to the local metrics_server. The options are: **twisted** (updates coalesced for reporting at 1 second intervals and done asynchronously), **synchronous** (all updates are reported immediately and socket operations may block), **off** (no relay)

log_storage - (DEFAULT: 0) A value of 1 enables logging of storage object store routines to silo.log

memory_limit_mb - (DEFAULT: 1024) Memory limit in MB for use by the main data pull process

num_storage_procs - (DEFAULT: number of logical CPUs on the system to a maximum of 16) Size of the storage process pool. Must be explicitly configured for pool sizes greater than 16.

num_storage_retries - (DEFAULT: 2) Number of times to attempt retries of storage objects that fail due to storage timeouts, deadlocks, and lock wait timeouts

outstanding_storage_tasks_limit - (DEFAULT: 100000) Number of storage objects that can be queued up for storage at any given point in time (including retry storage objects)

pull_block_size - (DEFAULT: 5000) Number of storage objects to pull in each block

remote_checkpoint_interval – (DEFAULT: 30) Number of minutes between flushes of remote checkpoints. Remote checkpoints are only used for table pruning, so this is a low priority task

storage_process_refresh_interval - (DEFAULT: 1000000) Number of storage tasks to execute before forcing a restart of a storage pool process

store_timeout - (DEFAULT: 60) Number of seconds before data pull will timeout a long running storage object

Testing

Unit Testing

Unit tests will be developed both for helper library code and for components dependent on Twisted.

Twisted comes with its own unit testing framework and runner called Trial. Trial is an alternative to Nose that is geared towards testing Twisted-based code. The runner uses its own reactor to run tests and to allow deferred verification.

The Mock library will be used to reduce dependencies on external resources.

The goals of the tests are to:

- Test application level logic in isolation
- Do not depend on external resources (database connections, etc.)
- Executable in an environment that does not require a live system.

QA Testing

This project aspires to facilitate the automation of the following types of tests against deployed appliances:

- Functional Testing
- Performance and Scale Testing

To help with that, this project will allow the tester to engage in grey-box testing by allowing the development of arbitrary storage object definitions and providing an input generator/injector. For functional testing, this will provide a control – a base install will not require that devices are discovered and that collection is occurring.

Verification of those tests may be automated or done manually. The unittest library can be leveraged for its test framework (even though these are not unit tests).

Functional Testing Strategy

Storage object timeout handling example: Create a storage object whose store() routine executes a sleep longer than the storage object timeout.

Verifying that all storage objects pulled are stored: Create storage objects that increment a counter in some sort of external data store (mysql table, file, etc.).

Performance and Scale Testing Strategy

Performance metrics will be made available for polling using the metrics_client.py tool.

Pull performance example: Create a storage object whose store() routine is a no-op.

Instrumentation

Introduction

Data pull uses the silo_metrics package for instrumentation. This provides a window into the operating state of data pull with minimal performance impact and without requiring debug log analysis.

silo_metrics provides facilities for defining a hierarchical schema for "metrics groups" – groups of reported metrics – and for relaying updates outside of the instrumented process.

To minimize the impact of instrumentation, EM7 uses an in-memory collector service (metrics_server) that listens on a UNIX Domain Socket. Instrumented processes communicate updates to this service. This service also allows point-in-time querying for metrics values with the /usr/local/bin/silometrics command line tool.

Monitoring solutions (dynamic apps) can be built from querying metrics_server as has been done by ScienceLogic support.

Optimization for Data Pull Project

As part of this data pull project, silo_metrics was also optimized. Documenting the design of silo_metrics is outside the scope of this document. Instead, here is a list of changes made:

- New "twisted" relay created for communicating instrumented metrics updates
    - Updates are coalesced and sent in bulk on a regular schedule to reduce communication
    - Always async – no updates of a metric will block data pull code
- metrics_server
    - Metrics groups are now indexed so that updates do not require full tree walks
    - Allows bulk "digest" updates to reduce communication

Polling for Metrics

A new silometrics command line tool is now available on EM7 systems at /usr/local/bin.

Basic Usage

silometrics <metrics_group>[.<optional path>]

Data Pull Metrics Groups

- hf_pull
- lf_pull
- mf_pull

Example Usage

1. Full walk of mf_pull group: silometrics mf_pull
2. Walk of the mf_pull.collector field: silometrics mf_pull.collector
3. Walk of only collector ID 3: silometrics mf_pull.collector[3]
4. Querying for the mysql_connection_id field for all collectors: silometrics mf_pull.collector[*].mysql_connection_id

Schema

This is a description of the metrics schema. Each bullet point is a field in a substructure.

lf_pull | mf_pull | hf_pull: The metrics group container for the specific data pull instance

- **proc**: main data pull process state
  - **pid**: process ID
  - **tid**: Python thread ID (the system thread ID will = pid)
  - **uid**: user ID
  - **gid**: group ID
  - **start_time**: time started
  - **mysql_connection_id**: Connection ID of the main mysql connection handed off by the proc_pda framework
- **config**: the configuration options as listed in the Configurables section
  - **<see Configurables section for list of options>**
- **tasks**: StorageTaskManager metrics related to the task buffer, task distribution, and storage results
  - **active**: Gauge, number of tasks distributed to the process pool
  - **processed**: Counter, number of tasks completed (including failures and each retry)
  - **outstanding**: Gauge, number of storage objects pulled and not yet completed (queued in the buffer or currently active). Limited by the outstanding_storage_tasks_limit configuration option.
  - **retries_outstanding**: Gauge, number of retries currently outstanding. Could be a sign of system distress.
  - **outstanding_reserved**: Gauge, outstanding tasks + reservations held by incomplete pull requests
  - **retried**: Counter, number of retry attempts completed

    - **dropped**: Counter, number of storage objects that were dropped due to errors and could not be successfully retried
    - **successes**: Counter, number of successful attempts
    - **timeouts**: Counter, number of storage objects that failed to complete within the store_timeout period. These tasks are subject to retries
    - **lock_wait_timeouts**: Counter, number of storage objects that failed due to MySQL LOCK_WAIT_TIMEOUT
    - **deadlocks**: Counter, number of storage objects that failed due to MySQL DEADLOCK
    - **errors**: Counter, number of storage objects that failed due to an unhandled exception raised in the store routine
    - **corrupted**: Counter, number of storage objects that could not be deserialized. Example: storage objects that are truncated in the in_storage table for being too big.
    - **connections_lost**: Counter, number of storage objects that failed due to the pool process connection being lost.
    - **fatal_errors**: Counter, number of storage objects that failed due fatal signals (such as segfaults). Could be an indication of a Python extension bug or an attempt to deserialize a pickled resource handle.
    - **unexpected_exits**: Counter, number of storage objects that failed because the pool process exited for an unknown reason. Example: someone SIGTERMing the pool process manually.
    - **communication_errors:** Counter, number of storage objects that failed due to communication errors with the process pool.
    - **unexplained_errors:** Counter, number of storage objects that failed for some reason not explained above.
    - **deferred_queues[]**: keyed by storage object serialization keys. Indexes are transient (can and will be popped). There can be up to num_storage_procs deferred queues. Deferred queues do not have a permanent relationship with a particular pool process.
        - **size**: Number of storage objects in the queue
        - **start_time**: Time when this serialization key was first claimed
- **pool**: Process pool metrics
    - **tasks**: Counter, number of tasks submitted to the process pool
    - **net_store_time**: Counter, total wall time in seconds for storage object execution
    - **queued_tasks**: Gauge, number of tasks queued in the pool and waiting to be submitted to a free storage process
    - **active_tasks**: Gauge, number of tasks currently assigned to a pool process
    - **ready_procs**: Gauge, number of idle pool processes
    - **dead_proc_counter**: Counter, number of pool processes that were queued as ready but became unavailable before task assignment. This is an abnormal condition.
- **pool_process[]**: Keyed by sequential integer ID for each pool process slot

- **pid**: Process ID
- **start_time**: Time the process started
- **mysql_connection_id**: ID for MySQL connection created for this pool process
- **total_tasks_counter**: Counter, number of tasks total for this process slot
- **cur_tasks_counter**: Counter, number of tasks for this current process instance for this slot, When the counter reaches the storage_process_refresh_interval limit, the pool process will be forcibly restarted.
- **restarts**: Counter, number of times this process was restarted
- **store_timeouts**: Counter, number of storage object timeouts initiated by data pull. These will result in a restart of the process.
- **init_timeouts:** Counter, number of process initialization timeouts. Indication of a heavily loaded system
- **retired**: Counter, number of times this process was restarted for exceeding the storage_process_refresh_interval limit
- **mysql_connections_lost**: Counter, number of times the process lost its mysql connection. This is an atypical situation that will result in a restart of the process.
- **exit_unexpected**: Counter, exits for unexpected reasons. Example: SIGTERM when none was issued by data pull
- **exit_fatal_signal**: Counter, exits due to fatal signals such as SIGSEGV.
- **exit_sigterm**: Counter, exits due to SIGTERMs (usually issued by data pull for timeout conditions)
- **exit_sigkill**: Counter, exits due to SIGKILLs (issued by data pull when no response to a SIGTERM – likely due to a blocking C extension call)
- **current_task**: Information about the task currently assigned to this process
  - **collector**: Origin collector
  - **seq_id**: Sequence ID from the collector's in_storage table
  - **type**: Class of the storage object
  - **did**: Device ID if applicable
  - **key**: Serialization key

- **pull**: CollectorManager metrics
  - **refreshes**: Counter, number of collector list refreshes. Should correspond to the frequency of the data pull instance.
  - **rows_pulled**: Counter, total number of rows pulled from all collectors
  - **bytes_pulled**: Counter, total size of serialized storage objects pulled in bytes from all collectors
  - **pulls_queued**: Gauge, number of pull requests from the StorageTaskManager currently queued. Limit: floor(outstanding_storage_tasks_limit/pull_block_size)
  - **pulls_active**: Gauge, number of collector pulls deferred to the collector thread pool
  - **blocks_pulled**: Counter, number of storage object blocks pulled

    - **blocks_complete**: Counter, number of storage object blocks where all storage objects have been attempted at least once
- **collector[]**: Keyed on collector appliance ID
    - **host_addr**: Host address for this collector
    - **mysql_connection_id**: ID for the remote mysql connection
    - **min_seq_id**: Lowest sequence ID in the in_storage table
    - **max_seq_id**: Highest sequence ID in the in_storage table
    - **next_start_seq**: The sequence ID used for the next pull
    - **connects**: Counter, number of successful connect operations
    - **syncs**: Counter, number of successful sync operations
    - **pulls**: Counter, number of successful pull operations
    - **checkpoints**: Counter, number of successful remote checkpoint operations
    - **net_connect_time**: Counter, total time in seconds spent on connects
    - **net_sync_time**: Counter, total time in seconds spent on syncs
    - **net_pull_time**: Counter, total time in seconds spent on pulls
    - **net_checkpoint_time**: Counter, total time in seconds spent on checkpoints
    - **last_connect_time**: Time interval in seconds for the last connect
    - **last_sync_time**: Time interval in seconds for the last sync
    - **last_pull_time**: Time interval in seconds for the last pull
    - **last_checkpoint_time**: Time interval in seconds for the last checkpoint
    - **rows_pulled**: Counter, total number of rows pulled
    - **bytes_pulled**: Counter, total size of serialized storage objects pulled in bytes
    - **estimated_rows_to_pull**: Gauge, estimated number of rows that remain to be pulled. Based on results from the last sync operation.
    - **connection_errors**: Counter, number of connection errors
    - **connections_lost**: Counter, number of connections lost
    - **operational_errors**: Counter, number of MySQL operational failures. Anything greater than zero is likely an indication of poor database health (for example: corrupt tables and partitions)
    - **unhandled_exceptions**: Counter, number of non-MySQL exceptions. Anything greater than zero indicates a bug/logic error.

Troubleshooting

- Before restarting data pull (to enable debugging, because it's "hanging", etc), capture the full output of silometrics output for that instance of data pull (example: >>`silometrics mf_pull`)
- When investigating overall data pull performance issues, use the silometrics instrumentation to narrow it down to being a pull or storage issue
- Storage Object failures that result in a storage object being dropped will be logged to System Logs

Miscellaneous

Open Issues

Queue Starvation – provision for aging priorities
Resource issue with multiple instances – single process for all data pull frequencies – different configurables for each priority/frequency
Consistency – Storage Process Protocol & Storage Process Proxy Protocol

Glossary

Central Checkpoint Table – The database table on the central database that maintains the checkpoint metadata (master.datapull_checkpoints)

Checkpoint – Metadata to track progress in pulling/storing storage objects from a collector's storage table. A checkpoint is not attempted until at least a single attempt is made to store each storage object within a storage block.

Collector Refresh – A step in Data Pull in which the list of collectors available for pulling is refreshed. This triggers a possible connect/re-connect followed by synchronization if we are currently in a "caught-up" state for that collector.

Deferred – An object that represents a promise that an action will be completed sometime in the future (most likely at a later iteration of the Twisted event loop). All asynchronous requests in Data Pull return a deferred. Clients of those async requests can then attach callbacks to these deferreds that will get fired on completion.

Deferred Queue – (not related to Twisted) A queue for serialized storage object tasks

Distributed System – An ScienceLogic EM7 system that comprises multiple appliances with specialized roles

Frequency – Minimum rate of polling collectors for storage objects. There are three frequencies each with a corresponding instance of data pull – High, Medium, Low.

Metrics – Internal application metrics that are tracked and reported using the silo_metrics library

Outstanding Tasks – The number of storage objects in data pull's queues at any given time.

Pickle – The Python serialization (persistence) format for Python objects.

Protocol Object – Protocol in the object-oriented sense – maintains state for a client of a service and enforces rules for interacting with the service. This is a pattern prevalent in the Twisted standard library.

Pull – The act of querying for storage objects queued up on collector databases

Reactor – Twisted's main event loop

Remote Checkpoint Table – The database table on the collector that maintains the checkpoint metadata (in_storage.data_storage_last_collect)

Storage Block – A contiguous block of storage objects atomically pulled from a collector.

Storage Object – A Python object that contains data that a collector unit wishes to communicate to the central database. Storage objects are queued in the storage table as Python pickles. The store routine defines how the central database will act on that data.

Storage Object Key – A key generated per storage object to enforce serialization (ordered operation) for a group of tasks that cannot be parallelized.

Storage Object Timeout – Application-level timeout for completion of a storage object's store() routine.

Storage Table – The database table on the collector that holds the storage object queue. There is a table for each frequency level. (in_storage.hf_storage, in_storage.mf_storage, in_storage.lf_storage)

Store – The act of executing a storage object's store routine

Store Routine – The store() method defined in each storage object class. This defines how the central database will act on the data contained by the storage object.

Synchronize – The act of querying for a collector's last checkpoint and min/max id counters in the storage table

Task – A chunk of work. The type of work is contextual. Tasks handed off to the storage pool are in the form of storage objects (state) and their store() routines.